

Introduction to Compilers

Problem Set: Automating Lexical Analysis (Lecture 3)

Instructions. Five problems, covering the full lecture: NFAs and the DFA state-count blowup, Thompson’s construction, subset construction, DFA minimization, and longest match. Estimated total time: 90–105 minutes. Show your work throughout — partial credit is available on every problem.

Problem 1 — NFAs, nondeterminism, and the DFA blowup (~25 min)

Consider the following NFA over $\{a, b\}$, recognizing all strings that contain “ aa ” as a substring:

$$\begin{array}{lcl} s_0 & \xrightarrow{a,b} & s_0 \quad (\text{self-loop}) \\ s_0 & \xrightarrow{a} s_1 \xrightarrow{a} s_2 & (\text{accept}) \\ s_2 & \xrightarrow{a,b} & s_2 \quad (\text{self-loop}) \end{array}$$

Note that s_0 has *two* outgoing transitions on a — this is genuine nondeterminism, with no ε -edges involved.

- (a) Trace an accepting path for **baab**, showing the sequence of nondeterministic choices made at each step.
 - (b) Explain why **abab** is rejected. Since this is an NFA, you must argue that *no* sequence of choices from s_0 reaches an accept state — not just that one particular path fails.
 - (c) Run subset construction on this NFA, starting from $\varepsilon\text{-closure}(\{s_0\}) = \{s_0\}$. List every reachable DFA state (as a set of NFA states), marking which are accepting, and give the transition table on a and b . Continue until no new DFA states appear.
 - (d) How many DFA states did you find? Compare this to the NFA’s 3 states, and briefly explain what this illustrates about subset construction in general.
-

Problem 2 — Thompson’s construction (~15 min)

Apply Thompson’s construction to the regular expression **(a|b)*c**, the same way it was applied to **(0|1)*1** in lecture.

- (a) List each rule applied, in order, and how many NFA states each one contributes.

- (b) How many states does the complete NFA have in total?
- (c) This NFA turns out to have the same state count as the $(0|1)^*1$ example from lecture. Explain, rule by rule, why the two constructions add up to the same total even though the regular expressions look different.
-

Problem 3 — Subset construction with ε -transitions

(~15 min)

Consider the following NFA over $\{x, y\}$, recognizing “ x ” or “ xy ”:

$$\begin{array}{lcl} r_0 \xrightarrow{\varepsilon} r_1 \xrightarrow{x} r_2 & & (\text{accept}) \\ r_0 \xrightarrow{\varepsilon} r_3 \xrightarrow{x} r_4 \xrightarrow{y} r_5 & & (\text{accept}) \end{array}$$

- (a) Compute ε -closure(r_0), and then the DFA transition on symbol x from that closure.
- (b) From the DFA state you just found, compute the transition on symbol y . Is the resulting state accepting?
- (c) From the same DFA state (after reading x), what happens on a *second* x ? Explain your answer in terms of which NFA states are reachable.
-

Problem 4 — DFA minimization

(~25 min)

Consider the following DFA over $\{0, 1\}$, recognizing strings that contain at least one 1:

	0	1
v_0 (start)	v_1	v_2
v_1	v_1	v_3
v_2 (accept)	v_2	v_2
v_3 (accept)	v_3	v_3

- (a) Build the distinguishability table for all $\binom{4}{2} = 6$ pairs of states, applying Rule 1 first, then sweeping with Rule 2 until nothing new is marked. Show your work for every pair.
- (b) Which states turn out to be equivalent? There is more than one equivalence class here — find all of them.
- (c) Give the transition table for the final minimized DFA.
-

Problem 5 — Longest match and rule order

(~15 min)

A lexer has a keyword rule `"class"` and an identifier rule matching `[a-zA-Z][a-zA-Z0-9_]*`.

- (a) Would the input `classroom` ever be tokenized as `CLASS` followed by `ID("room")`? Justify your answer using longest match.
- (b) Now suppose the input is exactly `class` (5 characters, nothing more). Does longest match alone resolve which rule fires? If not, what does, and which rule wins?
- (c) In general, under what circumstance does rule order ever actually get consulted? Answer in one sentence.